

Helix AI: Testing and Evaluation Report (In-Depth)

What was tested, how it was tested, and what results mean

Generated: 2026-04-05 01:24:58

This report presents the quality strategy for pipeline, API, and ML data assets, with a live snapshot of current test/evaluation status.

Project Snapshot	Value
Test files	3
Test methods	10
Unit test status	pass (10)
Rule benchmark pass	4/4 (100.0%)
Backend LOC	1158
Frontend LOC	2647
Test LOC	157
Detected endpoints	10
Migration transform rules	15
Rule benchmark pass	4/4
Unit test status	pass (10 tests)

How to read this document: each section starts with a simple explanation, then a deeper engineering explanation, then a practical checklist.

0. Test Strategy in Simple Terms

Simple Explanation

In simple words: we test the pipeline logic, API behavior, and ML assets separately so failures are easier to diagnose.

Deep Explanation

Current tests include about 10 explicit test methods across 3 test files.

Pipeline tests verify end-to-end transformation correctness on representative legacy snippets.

API tests ensure route contracts and response fields remain stable.

Practical Checklist

- Keep tests grouped by concern.
- Prefer deterministic fixtures.
- Verify both success and failure paths.

Code References

```
tests/test_pipeline.py
tests/test_api.py
tests/test_ml_assets.py
```

1. Pipeline Unit Tests

Simple Explanation

In simple words: these tests check if old patterns are detected, transformed, and validated correctly.

Deep Explanation

A core sample verifies modernization of print/xrange/has_key/raw_input in one flow.

A very-old sample verifies backtick repr and apply builtin handling for Python 1-era style.

Validation failure tests verify that leftover legacy constructs are rejected.

Practical Checklist

- Add one test per new transform rule.
- Add one negative test per risk area.
- Ensure deterministic expected output strings.

Code References

```
tests/test_pipeline.py
```

2. API Contract Tests

Simple Explanation

In simple words: API tests ensure frontend can trust backend response shapes.

Deep Explanation

Analyze endpoint test asserts availability of analysis, plan, and ML status blocks.

Refactor endpoint test checks for new code, diff output, and validation status.

Run-history tests verify saved run retrieval path.

Practical Checklist

- Pin response field names in tests.
- Cover health and model-status routes.
- Cover run list and run detail retrieval.

Code References

`tests/test_api.py`

3. ML Asset and Benchmark Tests

Simple Explanation

In simple words: ML tests confirm dataset presence and evaluator script behavior.

Deep Explanation

Dataset sanity test checks curated training data exists and has required fields.

Evaluator invocation test ensures modernization benchmark script returns expected structure.

This gives baseline confidence even before full model training runs.

Practical Checklist

- Validate data schema before training.
- Treat evaluator as regression gate.
- Add adapter-loading tests when model artifacts are present.

Code References

```
tests/test_ml_assets.py  
ml/evaluate_modernizer.py
```

4. Current Execution Snapshot

Simple Explanation

In simple words: this report includes a live test/eval snapshot generated from your current workspace.

Deep Explanation

Unit test run status: pass, discovered tests: 10.

Benchmark pass summary: 4/4 (100.0 percent).

If these numbers differ in your local run, regenerate this PDF to keep evidence aligned.

Practical Checklist

- Regenerate reports after major code changes.
- Keep test and benchmark outputs versioned for submissions.
- Use same environment when comparing metrics.

Code References

```
docs/generate_all_in_depth_reports.py
```

5. Failure Mode Coverage

Simple Explanation

In simple words: tests should also prove that unsafe outputs are blocked.

Deep Explanation

Validation layer is tested for rejection when legacy constructs remain after transformation.

Blocked/restricted analyzer modes should be exercised with dynamic execution samples.

Run-history retrieval should be tested for non-existent run IDs.

Practical Checklist

- Add explicit blocked-mode test cases.

- Add malformed-input API tests.
- Add concurrency tests for run-store growth scenarios.

Code References

core/validation.py
agents/analyzer.py
main.py

6. Quality Gates for Delivery

Simple Explanation

In simple words: code should be merged only when tests and benchmark checks pass.

Deep Explanation

Minimum gate can include pipeline tests, API tests, and evaluator smoke benchmark.

Optional gate can include style checks and static analysis.

Release notes should include transform changes and risk notes.

Practical Checklist

- Define mandatory checks in one script or CI job.
- Fail build on regression in critical tests.
- Track flaky tests and stabilize quickly.

Code References

```
tests/  
ml/evaluate_modernizer.py
```

7. Evaluation Roadmap

Simple Explanation

In simple words: to claim stronger quality, we need larger benchmark sets and per-pattern metrics.

Deep Explanation

Add stratified benchmark slices by syntax pattern type and risk category.

Track precision/recall for each transform rule using labeled samples.

Add human-review scoring rubric for readability and behavior trust.

Practical Checklist

- Scale benchmark beyond smoke-test size.
- Track per-rule failure heatmap.
- Keep monthly evaluation snapshots for trend analysis.

Code References

```
ml/data/eval_benchmark.jsonl  
docs/pdfs/
```

Viva Quick Answers

Q: What do your tests prove?

They prove detection, transformation, validation behavior, API contract stability, and evaluator execution.

Q: How do you handle regressions?

By adding rule-specific tests and using benchmark checks as release gates.

Q: Is benchmark enough today?

It is a useful smoke baseline, but needs expansion for stronger statistical confidence.

Q: Why include failure-path tests?

Because safe migration requires proving blocking behavior as much as success behavior.

Q: What is your next testing milestone?

Per-pattern metrics with larger benchmark sets and CI automation.

From Scratch Learning Track

This chapter is written for beginners first. Each module starts with very simple meaning, then engineering depth, then practical action.

Module 1: Test Pyramid For This Project

Simple view: test pyramid for this project explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, test pyramid for this project connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 2: Pipeline Functional Test Strategy

Simple view: pipeline functional test strategy explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, pipeline functional test strategy connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 3: Api Contract Test Strategy

Simple view: API contract test strategy explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, API contract test strategy connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 4: Dataset And Evaluator Checks

Simple view: dataset and evaluator checks explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, dataset and evaluator checks connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 5: Failure-Path Test Importance

Simple view: failure-path test importance explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, failure-path test importance connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 6: Benchmark Interpretation Limits

Simple view: benchmark interpretation limits explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, benchmark interpretation limits connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 7: Regression Prevention Workflow

Simple view: regression prevention workflow explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, regression prevention workflow connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 8: Quality Gate Setup

Simple view: quality gate setup explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, quality gate setup connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 9: Traceability Of Test Evidence

Simple view: traceability of test evidence explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, traceability of test evidence connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 10: Report Regeneration Discipline

Simple view: report regeneration discipline explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, report regeneration discipline connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 11: Coverage Expansion Roadmap

Simple view: coverage expansion roadmap explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, coverage expansion roadmap connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 12: CI-Focused Delivery Hardening

Simple view: CI-focused delivery hardening explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, CI-focused delivery hardening connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Hands-On Practical Labs

1. Lab 1: Baseline Input Audit

Start from a legacy snippet and identify exactly what will break on modern Python.

- Collect one legacy input sample.
- Run /analyze and record issue ids.
- Classify each issue as syntax or semantic risk.
- Write one paragraph on expected modernization output.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

2. Lab 2: Analyze Output Interpretation

Understand how analysis payload maps to later planner and execution actions.

- Inspect probable_source_version and mode.
- Read legacy_issues list line by line.
- Map each issue id to a suggestion id.
- Explain why some items should become warning.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

3. Lab 3: Critic + Planner Trace

Trace how safety score and priority become selected_plans.

- Run analyze output through critique logic.
- Compute final score manually for two suggestions.
- Verify execution order in planner output.
- Compare manual and program outputs.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

4. Lab 4: Execution and Diff Review

Run modernization and inspect unified diff before applying changes.

- Execute /refactor on sample input.
- Inspect added and removed lines.
- Confirm each line links to a planned transform.
- Document one risky change for manual review.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

5. Lab 5: Validation Failure Drill

Learn fail-closed behavior by forcing a leftover-legacy construct.

- Create code where one legacy construct remains.
- Run validation and capture failure stage.
- Explain rollback behavior and user impact.
- Fix rule and rerun to green state.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

6. Lab 6: Run History and Reproducibility

Use run history to prove the pipeline is auditable and repeatable.

- Generate at least two runs with different inputs.
- Use /runs and /runs/{id} endpoints.
- Re-open one run in UI and inspect report tab.
- Write traceability summary notes.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

7. Lab 7: Topic-Specific Deep Task

Perform a deeper exercise related to testing concerns and document engineering tradeoffs.

- Choose one module strongly tied to this topic.
- Read code and summarize control flow.
- List strengths, risks, and missing checks.
- Propose one measurable improvement.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

8. Lab 8: Evaluation and Evidence Pack

Prepare evidence artifacts you can use in submission and viva.

- Run tests and benchmark script.
- Capture summary metrics and limitations.
- Align report claims with actual metrics.
- Store outputs in a clean docs folder.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

9. Lab 9: Explain to Beginner

Re-explain one complex part in simple words without losing correctness.

- Pick one algorithmic decision.
- Write a beginner explanation in 8 lines.
- Write an expert explanation in 8 lines.
- Ensure both versions match same truth.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

10. Lab 10: Full Demo Rehearsal

Do complete end-to-end dry run as if in final presentation.

- Start server and open UI.
- Run one safe case and one warning case.
- Show diff, report, and run history.
- Conclude with roadmap and limitations.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

Troubleshooting Playbook

Issue	Likely Cause	How To Fix
Analysis returns no issues unexpectedly	Input pattern not covered by rules	Add/adjust analyzer rule and unit test.
Planner selects zero actions	Suggestions became NOOP or filtered by status	Inspect suggestion ids, critique map, and planner filters.
Validation fails after execution	Legacy leftovers or syntax problems remain	Inspect failure stage and patch transform logic.
Diff looks confusing to users	Output formatting or context lines are unclear	Improve diff rendering and report hints.
Run history missing expected data	RunStore payload fields not persisted	Verify run payload contract in save_run call.
ML shows unavailable	Adapter path missing or model disabled	Check env flags and adapter directory path.
UI button state feels wrong	State machine and action label out of sync	Trace state.primaryAction transitions.
Performance slows on large input	Heavy rendering or repeated processing	Paginate rendering and profile hot paths.
Benchmarks look too optimistic	Benchmark set too small or narrow	Expand dataset and stratify by pattern type.
Viva answers feel weak	No concrete evidence references	Quote tests, benchmark numbers, and code paths.
Topic-specific confusion in test fixtures	Overly generic to concrete file and flow	Add diagram plus file-level walkthrough.
Claims sound exaggerated	Scope and limitations not stated	Add explicit limitations and roadmap stages.

Glossary (Simple Words, Technical Meaning)

1. Legacy Pattern

An old syntax or API usage from earlier Python versions.

2. Modernization

Converting legacy code to current Python-compatible code.

3. Deterministic

Same input always gives same output.

4. Semantic Risk

Code may still run but behavior could change.

5. Validation Gate

Final checks that decide pass or rollback.

6. Diff-First Review

Inspect exact changes before applying to editor.

7. Analyzer

Stage that detects issues and risk indicators.

8. Suggester

Stage that maps detected issues to upgrades.

9. Critic

Stage that assigns safety scores and warnings.

10. Planner

Stage that chooses execution order.

11. Execution Core

Stage that applies selected transforms.

12. RunStore

Persistence layer for run history records.

13. LOCA (LOC)

Lines of code metric for rough project size.

14. Benchmark

Set of test cases used to measure pass rate.

15. Smoke Test

Small test set for quick health verification.

16. Rollback

Revert to original code when validation fails.

17. Adapter

LoRA fine-tuned parameter delta for base model.

18. Base Model

Original pretrained model used before fine-tuning.

19. Confidence

Estimated reliability signal for a suggestion.

20. Traceability

Ability to explain what changed and why.

21. Fixture

Controlled sample input used in tests.

22. Assertion

Condition that test expects to be true.

23. Regression

Previously fixed behavior breaking again.

24. Coverage Gap

Important behavior not yet tested.

25. Quality Gate

Required checks before release.

Extended Viva and Interview Q&A

Q1: What is the core goal of this report?

To explain this topic from beginner level to implementation depth with real project evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q2: How should I present this in viva?

Start with simple explanation, then show one concrete code path, then show one metric and one limitation.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q3: How do I avoid vague answers?

Always tie statements to file paths, endpoints, tests, or measured outputs.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q4: What if evaluator asks about limitations?

Answer directly and show roadmap with near-term measurable upgrades.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q5: How do I show project credibility?

Use deterministic workflow, validation behavior, and test/benchmark evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q6: Why use simple language in technical docs?

It improves clarity and comprehension without reducing technical correctness.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q7: How should I practice this topic?

Run one lab task, inspect code, and explain output in your own words.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q8: How does this topic connect to full system?

Each topic is one layer of the same end-to-end modernization pipeline.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q9: What does passing tests guarantee?

Core expected behaviors are preserved for tested patterns, not universal correctness for all code.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q10: Why still need manual review?

Semantic edge cases can exist beyond current test coverage.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q11: What test upgrade is highest impact?

Pattern-level benchmark expansion with failure category tracking.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Code Walkthrough Appendix (With Explanation)

This appendix connects explanation to real files. Each excerpt includes simple-language meaning, technical interpretation, and why it matters before showing code lines.

Excerpt 1: Pipeline Tests

File: /Users/rutwikbhondave/Desktop/Helix AI Project/tests/test_pipeline.py

Simple explanation: this block handles automated regression and contract verification. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=6, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: `legacy_scan`, `legacy_formatter`, `test_analyzer_detects_legacy_python_patterns`, `test_pipeline_modernizes_legacy_code`.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import asyncio
0002: import unittest
0003:
0004: from agents.analyzer import AnalyzerAgent
0005: from agents.critic import CriticAgent
0006: from agents.planner import PlannerAgent
0007: from agents.suggester import SuggestionAgent
0008: from core.execution import ExecutionCore
0009: from core.validation import ValidationCore
0010:
0011:
0012: LEGACY_SAMPLE = """def legacy_scan(values):
0013:     print "starting migration"
0014:     for i in xrange(len(values)):
0015:         if values.has_key(i):
0016:             print values[i]
0017:     return raw_input("done? ")
0018: """
0019:
0020: VERY_OLD_SAMPLE = """def legacy_formatter(value, args):
0021:     print `value`
0022:     return apply(str, args)
0023: """
0024:
0025:
0026: class PipelineTests(unittest.TestCase):
0027:     def test_analyzer_detects_legacy_python_patterns(self):
0028:         analysis = asyncio.run(AnalyzerAgent().process(LEGACY_SAMPLE))
0029:
0030:         self.assertTrue(analysis["success"])
0031:         self.assertEqual(analysis["probable_source_version"], "Python 2.x / early legacy Python")
0032:         self.assertGreaterEqual(len(analysis["legacy_issues"]), 4)
0033:         issue_ids = {issue["id"] for issue in analysis["legacy_issues"]}
0034:         self.assertIn("print_statement", issue_ids)
0035:         self.assertIn("xrange_usage", issue_ids)
```

```

0036:         self.assertIn("has_key_usage", issue_ids)
0037:         self.assertIn("raw_input_usage", issue_ids)
0038:
0039:     def test_pipeline_modernizes_legacy_code(self):
0040:         analyzer = AnalyzerAgent()
0041:         suggester = SuggestionAgent()
0042:         critic = CriticAgent()
0043:         planner = PlannerAgent()
0044:         executor = ExecutionCore()
0045:         validator = ValidationCore()
0046:
0047:         analysis = asyncio.run(analyzer.process(LEGACY_SAMPLE))
0048:         suggestions = asyncio.run(suggester.process(analysis, LEGACY_SAMPLE))
0049:         critiques = asyncio.run(critic.process(suggestions))
0050:         plan = asyncio.run(planner.process(suggestions, critiques))
0051:
0052:         modernized = executor.apply_changes(LEGACY_SAMPLE, plan["selected_plans"])
0053:         validation = validator.validate(LEGACY_SAMPLE, modernized)
0054:
0055:         self.assertIn("print(\"starting migration\")", modernized)
0056:         self.assertIn("range(len(values))", modernized)
0057:         self.assertIn("i in values", modernized)
0058:         self.assertIn("input(\"done? \")", modernized)
0059:         self.assertTrue(validation["success"])
0060:
0061:     def test_validation_rejects_leftover_legacy_constructs(self):
0062:         validator = ValidationCore()
0063:         result = validator.validate(
0064:             "for i in xrange(3):\n    print i\n",
0065:             "for i in xrange(3):\n    print(i)\n",
0066:         )
0067:
0068:         self.assertFalse(result["success"])
0069:         self.assertEqual(result["stage"], "LINT")
0070:         self.assertIn("xrange", result["error"])
0071:
0072:     def test_pipeline_handles_very_old_legacy_patterns(self):
0073:         analyzer = AnalyzerAgent()
0074:         suggester = SuggestionAgent()
0075:         critic = CriticAgent()
0076:         planner = PlannerAgent()
0077:         executor = ExecutionCore()
0078:         validator = ValidationCore()
0079:
0080:         analysis = asyncio.run(analyzer.process(VERY_OLD_SAMPLE))
0081:         self.assertTrue(analysis["success"])
0082:         self.assertEqual(analysis["probable_source_version"], "Python 1.x / early legacy Python")
0083:         issue_ids = {issue["id"] for issue in analysis["legacy_issues"]}
0084:         self.assertIn("backtick_repr", issue_ids)
0085:         self.assertIn("apply_builtin", issue_ids)
0086:
0087:         suggestions = asyncio.run(suggester.process(analysis, VERY_OLD_SAMPLE))
0088:         critiques = asyncio.run(critic.process(suggestions))
0089:         plan = asyncio.run(planner.process(suggestions, critiques))
0090:
0091:         modernized = executor.apply_changes(VERY_OLD_SAMPLE, plan["selected_plans"])
0092:         validation = validator.validate(VERY_OLD_SAMPLE, modernized)
0093:
0094:         self.assertIn("print(repr(value))", modernized)
0095:         self.assertIn("str(*args)", modernized)
0096:         self.assertTrue(validation["success"])
0097:
0098:
0099: if __name__ == "__main__":
0100:     unittest.main()

```

Excerpt 2: API Tests

File: /Users/rutwikbhondave/Desktop/Helix AI Project/tests/test_api.py

Simple explanation: this block handles automated regression and contract verification. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=5, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: setUpClass, test_analyze_endpoint_returns_plan_and_ml_status, test_refactor_endpoint_returns_diff_and_validation, test_health_and_model_status_endpoints.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import unittest
0002:
0003: from fastapi.testclient import TestClient
0004:
0005: from main import app
0006:
0007:
0008: LEGACY_SAMPLE = """try:
0009:     risky()
0010: except ValueError, exc:
0011:     print exc
0012: """
0013:
0014:
0015: class ApiTests(unittest.TestCase):
0016:     @classmethod
0017:     def setUpClass(cls):
0018:         cls.client = TestClient(app)
0019:
0020:     def test_analyze_endpoint_returns_plan_and_ml_status(self):
0021:         response = self.client.post("/analyze", json={"code": LEGACY_SAMPLE})
0022:         self.assertEqual(response.status_code, 200)
0023:
0024:         payload = response.json()
0025:         self.assertIn("analysis", payload)
0026:         self.assertIn("plan", payload)
0027:         self.assertIn("ml_reasoner", payload)
0028:         self.assertTrue(payload["analysis"]["success"])
0029:         self.assertEqual(payload["analysis"]["probable_source_version"], "Python 2.x / early legacy")
0030:
0031:     def test_refactor_endpoint_returns_diff_and_validation(self):
0032:         response = self.client.post("/refactor", json={"code": LEGACY_SAMPLE})
0033:         self.assertEqual(response.status_code, 200)
0034:
0035:         payload = response.json()
0036:         self.assertIn("new_code", payload)
0037:         self.assertIn("diff", payload)
0038:         self.assertIn("validation", payload)
0039:         self.assertIn("except ValueError as exc", payload["new_code"])
0040:         self.assertIn("print(exc)", payload["new_code"])
0041:         self.assertTrue(payload["validation"]["success"])
```

```

0042:
0043:     def test_health_and_model_status_endpoints(self):
0044:         health = self.client.get("/health")
0045:         model_status = self.client.get("/model-status")
0046:
0047:         self.assertEqual(health.status_code, 200)
0048:         self.assertEqual(model_status.status_code, 200)
0049:         self.assertEqual(health.json()["status"], "ok")
0050:         self.assertIn("available", model_status.json())
0051:
0052:     def test_runs_endpoints_return_stored_history(self):
0053:         refactor = self.client.post("/refactor", json={"code": LEGACY_SAMPLE})
0054:         self.assertEqual(refactor.status_code, 200)
0055:         run_id = refactor.json()["run_id"]
0056:
0057:         runs = self.client.get("/runs?limit=5")
0058:         single = self.client.get(f"/runs/{run_id}")
0059:
0060:         self.assertEqual(runs.status_code, 200)
0061:         self.assertEqual(single.status_code, 200)
0062:         self.assertTrue(any(item["run_id"] == run_id for item in runs.json()["runs"]))
0063:         self.assertEqual(single.json()["run_id"], run_id)
0064:
0065:
0066: if __name__ == "__main__":
0067:     unittest.main()

```

Excerpt 3: ML Asset + Evaluator

File: /Users/rutwikbhondave/Desktop/Helix AI Project/tests/test_ml_assets.py

Simple explanation: this block handles automated regression and contract verification. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=2, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: test_curated_dataset_exists_and_has_examples, test_evaluator_script_runs.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import json
0002: import subprocess
0003: import sys
0004: import unittest
0005: from pathlib import Path
0006:
0007:
0008: ROOT = Path(__file__).resolve().parent.parent
0009:
0010:
0011: class MLAssetTests(unittest.TestCase):
0012:     def test_curated_dataset_exists_and_has_examples(self):
0013:         dataset_path = ROOT / "ml/data/modernization_train.jsonl"
0014:         self.assertTrue(dataset_path.exists())
0015:
0016:         with dataset_path.open(encoding="utf-8") as handle:
0017:             rows = [json.loads(line) for line in handle if line.strip()]
0018:
0019:         self.assertGreater(len(rows), 50)
0020:         self.assertTrue(all("instruction" in row and "input" in row and "output" in row for row in rows))
0021:
0022:     def test_evaluator_script_runs(self):
0023:         process = subprocess.run(
0024:             [sys.executable, "ml/evaluate_modernizer.py"],
0025:             cwd=ROOT,
0026:             capture_output=True,
0027:             text=True,
0028:             check=True,
0029:         )
0030:         payload = json.loads(process.stdout)
0031:         self.assertEqual(payload["benchmark_size"], 4)
0032:         self.assertEqual(payload["rule_pass"], 4)
0033:
0034:
0035: if __name__ == "__main__":
0036:     unittest.main()
```

Excerpt 4: Orchestration Context

File: /Users/rutwikbhondave/Desktop/Helix AI Project/main.py

Simple explanation: this block handles API orchestration and full modernization flow control. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=4, classes=0, endpoint decorators=2.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: analyze_code, refactor_code, analyze_code, refactor_code.
- Endpoint decorators in excerpt: app.post, app.post.
- Risk/warning language exists in this block, so it affects safety signaling.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0180:             logging.error("Failed to read inquiries: %s", exc)
0181:             inquiries = []
0182:
0183:             inquiries.append(inquiry_data)
0184:
0185:             with open(file_path, "w", encoding="utf-8") as file:
0186:                 json.dump(inquiries, file, indent=4)
0187:
0188:             return JsonResponse(content={"success": True, "message": "Migration inquiry received."})
0189:     except Exception as exc:
0190:         logging.error("Form submission failed: %s", exc)
0191:         raise HTTPException(status_code=500, detail="Transmission failed")
0192:
0193:
0194: @app.post("/analyze")
0195: async def analyze_code(request: CodeRequest):
0196:     try:
0197:         analysis = await analyzer.process(request.code)
0198:         suggestions = await suggester.process(analysis, request.code)
0199:         critiques = await critic.process(suggestions)
0200:         plan_result = await planner.process(suggestions, critiques)
0201:         ml_status = reasoner.explain_status()
0202:
0203:         return JsonResponse(
0204:             content={
0205:                 "analysis": analysis,
0206:                 "suggestions": suggestions,
0207:                 "critiques": critiques,
0208:                 "plan": plan_result,
0209:                 "ml_reasoner": ml_status,
0210:             }
0211:         )
0212:     except Exception as exc:
0213:         logging.error("Analysis failed: %s", exc)
0214:         raise HTTPException(status_code=500, detail=str(exc))
0215:
0216:
0217: @app.post("/refactor")
0218: async def refactor_code(request: CodeRequest):
```

```

0219:     try:
0220:         logs = ["--- Starting Legacy Python Modernization Pipeline ---"]
0221:         run_id = None
0222:
0223:         analysis = await analyzer.process(request.code)
0224:         if not analysis.get("success"):
0225:             return JSONResponse(content={"error": "Analysis failed", "details": analysis.get("error")})
0226:
0227:         logs.append(f"Analyzer: probable source version = {analysis.get('probable_source_version')}")
0228:         logs.append(f"Analyzer: detected {len(analysis.get('legacy_issues', []))} legacy issues.")
0229:
0230:         if analysis.get("mode") == "BLOCKED":
0231:             logs.append("Execution blocked due to critical dynamic execution risk.")
0232:             return JSONResponse(content={"error": "Blocked", "logs": logs, "analysis": analysis})
0233:
0234:         suggestions = await suggester.process(analysis, request.code)
0235:         critique = await critic.process(suggestions)
0236:         plan_result = await planner.process(suggestions, critique)
0237:         selected_plans = plan_result.get("selected_plans", [])
0238:         ml_result = None
0239:
0240:         logs.append(f"Suggester: generated {len(suggestions)} migration suggestions.")
0241:         logs.append(f"Planner: selected {len(selected_plans)} transformations for execution.")
0242:
0243:         new_code = request.code
0244:         validation_result = {"success": False, "stage": "SKIPPED", "message": "No changes applied"}
0245:         validation_success = False
0246:
0247:         if selected_plans:
0248:             new_code = executor.apply_changes(request.code, selected_plans)
0249:             logs.append("Execution: transformations applied sequentially.")
0250:
0251:             validation_result = validator.validate(request.code, new_code)
0252:             if validation_result["success"]:
0253:                 validation_success = True
0254:                 logs.append("Validation: modernization passed all configured checks.")
0255:             else:
0256:                 logs.append(f"Validation failed: {validation_result.get('error')}")
0257:                 logs.append("Rollback: reverting to original code.")
0258:                 new_code = request.code
0259:         else:
0260:             logs.append("Planner: no executable transformations were selected.")
0261:
0262:         if reasoner.is_available():
0263:             ml_result = reasoner.modernize(request.code)
0264:             if ml_result.available and ml_result.output:
0265:                 logs.append("ML Reasoner: generated auxiliary modernization output.")
0266:             elif ml_result.error:
0267:                 logs.append(f"ML Reasoner unavailable: {ml_result.error}")
0268:
0269:         diff = "\n".join(
0270:             difflib.unified_diff(
0271:                 request.code.splitlines(),
0272:                 new_code.splitlines(),
0273:                 fromfile="legacy.py",
0274:                 tofile="modernized.py",
0275:                 lineterm="",
0276:             )
0277:         )
0278:
0279:         report = reporter.generate_report(
0280:             analysis=analysis,

```